CONFIDENTIAL — INTERNAL REPORT

Luminedge CRM

Comprehensive Project Architecture Report

Full-stack Customer Relationship Management system built with the MERN stack — managing leads, students, courses, mock tests, exams, and internal operations for Luminedge.

Prepared	Stack	Version	License
June 23, 2026	**MERN + TypeScript**	**1.0.0**	**MIT**

Table of Contents

01 Executive Summary

02 Project Statistics at a Glance

03 Technology Stack

04 System Architecture

05 Project Structure

06 Data Models (Mongoose Schemas)

07 API Surface — 19 Route Groups

08 Authentication & Authorization

09 Realtime Layer (Socket.IO)

10 Security Measures

11 Deployment Topology

12 Key Observations & Recommendations

1 Executive Summary

The **Luminedge CRM** is a purpose-built, full-stack Customer Relationship Management platform designed for **Luminedge**, an education and test-preparation company. Unlike generic CRM solutions, it is tailored to manage the complete lifecycle of student engagement — from initial lead capture through course enrollment, mock test scheduling, exam booking, university application processing, and ongoing communication.

The system is built on the **MERN stack** (MongoDB, Express.js, React/Next.js, Node.js) with **TypeScript** throughout both frontend and backend, ensuring type safety and developer productivity. It serves **three distinct user roles** — Admins, Business Development Managers (BDMs), and Students — each with a dedicated portal featuring role-specific dashboards, workflows, and data access controls.

The platform includes **19 API route groups**, **16 database models**, realtime push notifications via Socket.IO, built-in chat messaging, document management with cloud storage, client-side PDF and Excel report generation, and a comprehensive analytics dashboard.

> ℹ The application is currently deployed with the frontend on **Netlify**, backend on **Render.com** (always-on Node process for Socket.IO), database on **MongoDB Atlas**, and file storage on **Vercel Blob**.

---

2 Project Statistics at a Glance

19	16	19	3
API ROUTE GROUPS	DATA MODELS	CONTROLLERS	USER PORTALS
4	**3**	**14+**	**2**
BACKEND SERVICES	AUTH MIDDLEWARES	COMPONENT GROUPS	JWT TOKEN TYPES

3 Technology Stack

Frontend

Technology	Version	Purpose
Next.js	14.x	React framework with App Router (SSR/SSG)
React	18.2	UI component library
TypeScript	5.3	Type-safe development
Tailwind CSS	3.4	Utility-first CSS framework
DaisyUI	5.5	Tailwind component library
Framer Motion	12.x	Page & component animations
Chart.js + react-chartjs-2	4.5 / 5.3	Chart visualizations
Recharts	3.7	Additional chart visualizations
Axios	1.13	HTTP client for API calls
Socket.IO Client	4.8	Realtime WebSocket connection
react-beautiful-dnd	13.1	Drag-and-drop (Kanban boards)
react-hook-form	7.49	Form state management
jsPDF + jspdf-autotable	4.0 / 5.0	Client-side PDF generation
exceljs / xlsx	4.4 / 0.18	Excel report export
Lucide React + React Icons	—	Icon libraries
jose / jwt-decode	—	JWT handling (client-side)
@react-oauth/google	0.13	Google OAuth integration

Backend

Technology	Version	Purpose
Node.js	18+	JavaScript runtime
Express	4.18	Web framework / REST API
TypeScript	5.3	Type-safe development

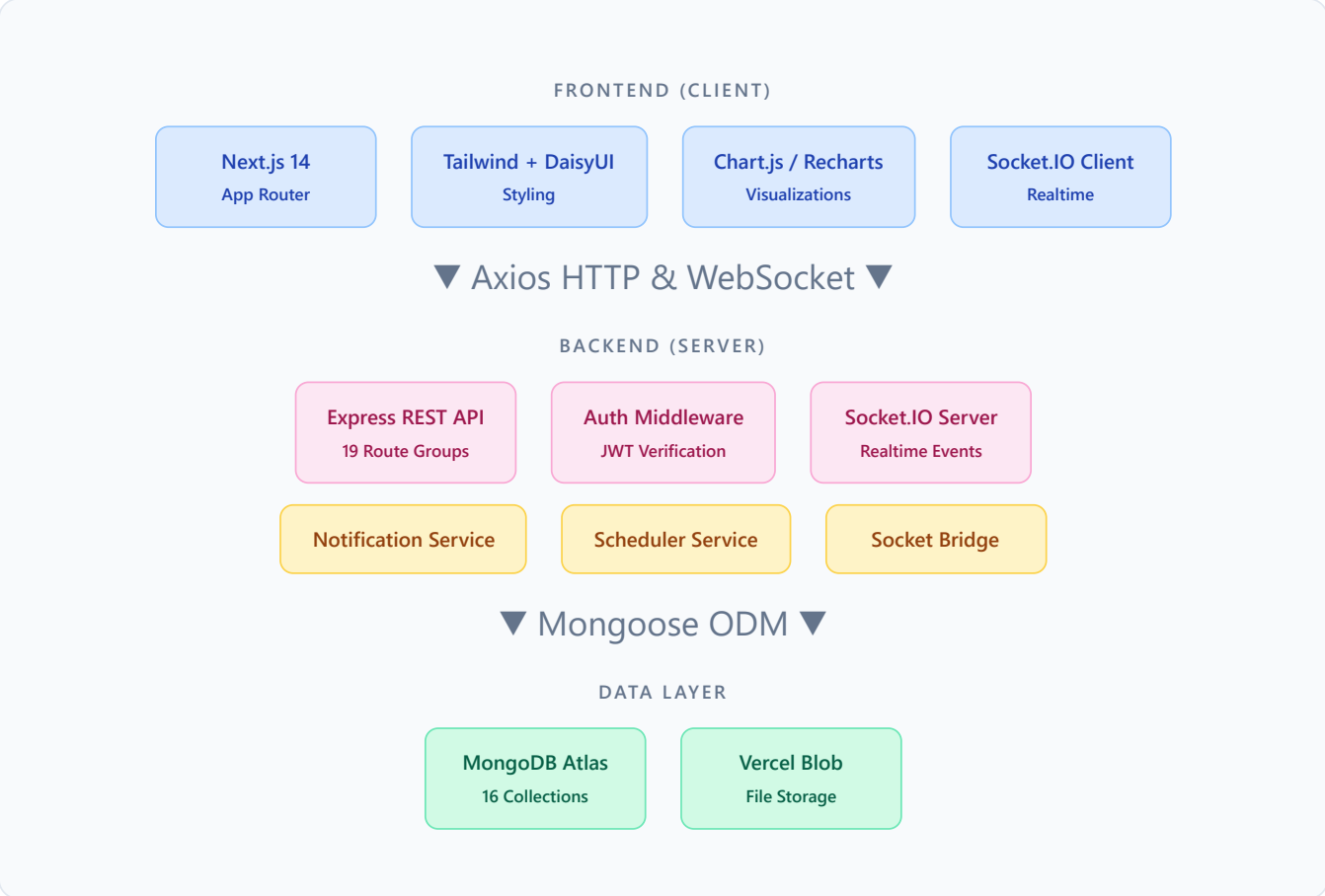
Technology	Version	Purpose
Mongoose	8.0	MongoDB ODM
Socket.IO	4.8	Realtime WebSocket server
jsonwebtoken	9.0	JWT signing & verification
bcryptjs	2.4	Password hashing
google-auth-library	10.5	Google OAuth verification
Nodemailer	6.9	Email delivery (SMTP)
Helmet	8.1	Security HTTP headers
express-rate-limit	8.3	Rate limiting
Multer	1.4	File upload handling
@vercel/blob	2.4	Cloud file storage
sanitize-html	2.17	XSS protection
PDFKit / pdf-parse	0.17 / 1.1	Server-side PDF generation & parsing
node-cron / node-schedule	3.0 / 2.1	Scheduled tasks

Testing

Technology	Purpose
Jest 30	Test runner & assertion framework
Supertest 7	HTTP endpoint testing
mongodb-memory-server	In-memory MongoDB for isolated tests
ts-jest	TypeScript preprocessing for Jest

4 System Architecture

The system follows a classic **client-server architecture** with a decoupled frontend and backend communicating over REST APIs and WebSocket connections.



Request Flow



5 Project Structure

Backend — `backend/src/`

```
backend/src/
├── app.ts                # Express setup, routes
├── server.ts             # HTTP + Socket.IO boot
├── config/               # DB connection
├── models/               # 16 Mongoose schemas
│   ├── User.ts
│   ├── Student.ts
│   ├── Lead.ts
│   ├── Application.ts
│   ├── Course.ts
│   ├── Exam.ts
│   ├── Task.ts
│   ├── Document.ts
│   ├── Notification.ts
│   ├── ... (7 more)
├── controllers/          # 19 controllers
├── routes/               # 19 route files
├── middleware/
│   ├── auth.middleware.ts
│   ├── studentAuth.middleware.ts
│   └── upload.middleware.ts
├── services/
│   ├── notification.service.ts
│   ├── socket.service.ts
│   ├── socket-bridge.service.ts
│   └── scheduler.service.ts
├── utils/
├── scripts/
└── __tests__/
```

Frontend — `frontend/`

```
frontend/
├── app/                  # App Router
│   ├── admin/            # Admin portal
│   │   ├── dashboard/
│   │   └── users/
│   ├── bdm/              # BDM portal
│   │   └── dashboard/
│   ├── student/          # Student portal
│   │   ├── dashboard/
│   │   ├── login/
│   │   └── register/
│   ├── dashboard/        # Legacy shared
│   ├── login/
│   ├── register/
│   ├── forget-password/
│   ├── reset-password/
│   ├── privacy/
│   └── terms/
├── components/           # 14+ groups
│   ├── admin/
│   ├── bdm/
│   ├── student/
│   ├── dashboard/
│   ├── leads/
│   ├── courses/
│   ├── layout/
│   ├── shared/
│   └── ...
├── lib/
│   ├── api/
│   ├── helpers/
│   ├── hooks/
│   └── socket.ts
└── middleware.ts         # Auth guards
```

6 Data Models (Mongoose Schemas)

The application uses **16 Mongoose models** stored in MongoDB. The models cover user management, education workflows, communication, and operational tracking.

Model	File	Size	Domain	Description
User	User.ts	1.6 KB	AUTH	Staff accounts (admin, BDM) with hashed passwords
Student	Student.ts	4.0 KB	CORE	Student profiles — the largest model with extensive fields
Lead	Lead.ts	2.9 KB	SALES	Sales pipeline leads with status tracking
Application	Application.ts	2.2 KB	CORE	University/course application records
Appointment	Appointment.ts	1.5 KB	OPS	Scheduled appointments between staff & students
Course	Course.ts	1.6 KB	CORE	Course catalog with details and pricing
Exam	Exam.ts	1.3 KB	CORE	Exam type definitions (IELTS, PTE, etc.)
MockTestPackage	MockTestPackage.ts	0.9 KB	CORE	Bundled mock test packages for purchase
MockTestSchedule	MockTestSchedule.ts	1.1 KB	OPS	Scheduled mock test sessions
Task	Task.ts	1.5 KB	OPS	Internal tasks with assignee and status
Milestone	Milestone.ts	2.3 KB	CORE	Student progress milestones & achievements
Document	Document.ts	2.2 KB	OPS	Uploaded files (passports, transcripts, etc.)
Conversation	Conversation.ts	1.2 KB	CHAT	Chat conversation threads
Message	Message.ts	0.9 KB	CHAT	Individual chat messages within conversations

Model	File	Size	Domain	Description
Notification	Notification.ts	1.1 KB	CHAT	Push notifications persisted for history
NewsUpdate	NewsUpdate.ts	1.0 KB	OPS	Announcements / news updates

7 API Surface — 19 Route Groups

All API endpoints are mounted under `/api` in the Express application. The heaviest controller is **dashboard.controller.ts** at 38 KB, handling complex analytics aggregations.

Route Prefix	Controller	Size	Domain
`/api/auth`	auth.controller.ts	23 KB	Login, register, password reset, Google OAuth
`/api/dashboard`	dashboard.controller.ts	38 KB	Analytics, metrics, charts, reports
`/api/leads`	leads.controller.ts	15 KB	Lead CRUD, pipeline, assignment
`/api/student`	student.controller.ts	15 KB	Student CRUD, profile, auth
`/api/reports`	reports.controller.ts	8 KB	Data export & reporting
`/api/tasks`	tasks.controller.ts	3 KB	Task management (Kanban)
`/api/courses`	courses.controller.ts	2.3 KB	Course catalog CRUD
`/api/mock-tests`	mockTests.controller.ts	4.5 KB	Mock test scheduling
`/api/mock-test-packages`	mockTestPackages.controller.ts	5.5 KB	Test bundle management
`/api/exams`	exams.controller.ts	2.3 KB	Exam definitions
`/api/schedules`	schedules.controller.ts	0.8 KB	General schedule queries
`/api/notifications`	notification.controller.ts	2.5 KB	Notification CRUD & read status
`/api/activities`	activities.controller.ts	3.8 KB	Activity log & audit trail
`/api/...` (apps)	application.controller.ts	6.2 KB	Student application processing
`/api/...` (chat)	chat.controller.ts	2.9 KB	Chat / messaging system
`/api/...` (docs)	document.controller.ts	3.0 KB	Document upload & management

Route Prefix	Controller	Size	Domain
`/api/ ...` (miles.)	milestone.controller.ts	1.5 KB	Student progress tracking
`/api/ ...` (news)	news.controller.ts	1.8 KB	News & announcements
`/api/ ...` (appts)	appointment.controller.ts	2.0 KB	Appointment booking

📌 A `/api/health` endpoint exists outside of rate limiting for Render.com health checks, returning `{"status": "OK"}`.

8 Authentication & Authorization

Dual Token System

The CRM separates staff and student authentication using **two distinct JWT cookies**, preventing token confusion across different user types.

Cookie Name	User Type	Roles	Issued By
`accessToken`	Staff	ADMIN BDM	`/api/auth/login`
`studentToken`	Student	STUDENT	`/api/student/auth/login`

Authentication Flow



Route Protection Matrix

URL Pattern	Required Role	Token	Enforced By
`/admin/*`	ADMIN	accessToken	Frontend middleware + backend
`/bdm/dashboard/*`	BDM	accessToken	Frontend middleware + backend
`/student/dashboard/*`	STUDENT	studentToken	Frontend middleware + backend
`/dashboard/*`	admin / bdm	accessToken	Frontend middleware + backend

> ⚠️ **Security Note:** The frontend Next.js middleware performs JWT *decode only* (no signature verification) — it is purely a UX routing layer. All actual data access authorization is enforced server-side by the backend auth middleware which performs full JWT signature verification.

Additional Auth Features

- **Google OAuth** — Staff can sign in with Google via `google-auth-library`
- **Password Reset** — Email-based flow using Nodemailer over SMTP
- **Rate Limiting** — Auth endpoints limited to 20 requests per 15 minutes
- **Token Expiry** — JWT tokens include `exp` claim; expired tokens are rejected

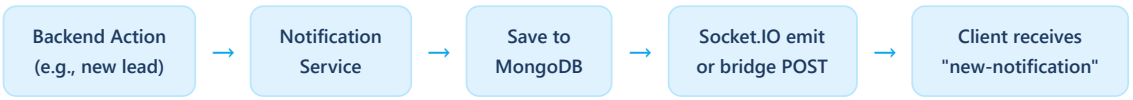
9 Realtime Layer (Socket.IO)

The system uses **Socket.IO 4.8** for bidirectional real-time communication, primarily for push notifications and chat messaging.

Backend Services

Service	File	Responsibility
Socket Service	socket.service.ts (5.9 KB)	Socket.IO server — manages connections, rooms, user mapping
Notification Service	notification.service.ts (2.7 KB)	Creates notifications in DB, triggers socket emission
Socket Bridge	socket-bridge.service.ts (1.3 KB)	Forwards events via HTTP when API and socket are on separate hosts
Scheduler Service	scheduler.service.ts (2.5 KB)	Cron-based background tasks (e.g., reminders)

Notification Flow



The socket bridge design allows a **split deployment** where the API runs on serverless (Vercel) while Socket.IO runs on an always-on process (Render) — communicating via an authenticated internal HTTP endpoint.

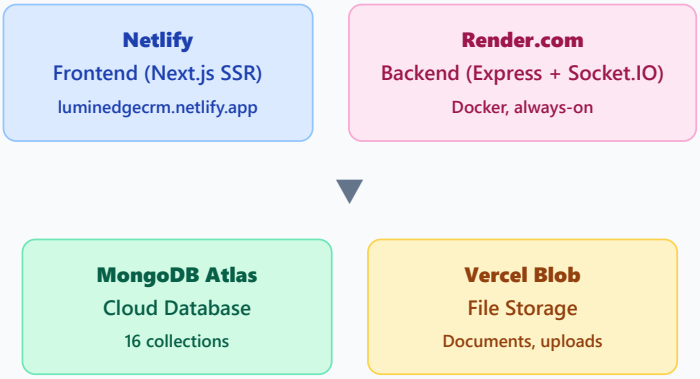
10 Security Measures

Layer	Technology	Configuration
HTTP Headers	Helmet 8.1	Default security headers (X-Content-Type-Options, X-Frame-Options, etc.)
CORS	cors middleware	Configurable allowed origins via `ALLOWED_ORIGINS` env var; credentials enabled
Rate Limiting	express-rate-limit	Auth endpoints: **20 req / 15 min**; General API: **2000 req / 15 min**
Password Hashing	bcryptjs	Salted bcrypt hashing for all stored passwords
Input Sanitization	sanitize-html	XSS protection on user-submitted HTML content
Request Parsing	Express body-parser	JSON and URL-encoded body size limited to **1 MB**
Proxy Trust	Express trust proxy	Set to `1` for proper IP detection behind reverse proxies
Socket Bridge Auth	Shared secret header	Inter-service calls authenticated via `x-socket-server-secret`
File Upload	Multer	Server-side file validation with upload middleware

Rate Limit Targets

```
/api/auth/login            → 20 req / 15 min
/api/auth/register         → 20 req / 15 min
/api/auth/forget-password   → 20 req / 15 min
/api/auth/reset-password    → 20 req / 15 min
/api/auth/set-password      → 20 req / 15 min
/api/student/auth/forgot-password → 20 req / 15 min
/api/student/auth/reset-password  → 20 req / 15 min
/api/* (general)            → 2000 req / 15 min (dashboard excluded)
```

11 Deployment Topology



Environment Variables

Backend (Render)

- `MONGODB_URI`
- `JWT_SECRET`
- `GOOGLE_CLIENT_ID`
- `ALLOWED_ORIGINS`
- `FRONTEND_URL`
- `BLOB_READ_WRITE_TOKEN`
- `SMTP_HOST / PORT / USER / PASS / FROM`
- `NODE_ENV=production`

Frontend (Netlify)

- `NEXT_PUBLIC_API_URL`
- `NEXT_PUBLIC_SOCKET_URL`

> 📦 A `render.yaml` blueprint is provided for one-click Render deployment with health check at `/api/health` and auto-deploy enabled.

12 Key Observations & Recommendations

Strengths

#	Observation
1	**Domain-specific design** — Purpose-built for education CRM rather than a generic solution, with deep integration of student lifecycle workflows.
2	**Full TypeScript** — Both frontend and backend use TypeScript, improving code quality, refactoring safety, and developer experience.
3	**Dual token separation** — Staff and student auth use separate cookies, preventing token confusion and simplifying role-based access.
4	**Flexible deployment** — Socket bridge pattern allows API and WebSocket server to run on the same or different hosts.
5	**Comprehensive security** — Helmet, rate limiting, CORS, bcrypt, input sanitization, and request size limits all properly configured.
6	**Rich dashboard analytics** — The 38 KB dashboard controller indicates extensive reporting and KPI tracking capabilities.
7	**Test infrastructure** — Jest + Supertest + mongodb-memory-server provides a solid foundation for automated testing.

Areas for Consideration

#	Observation	Impact
1	**Dashboard controller complexity** — At 38 KB, dashboard.controller.ts likely contains complex aggregation logic that could benefit from being split into focused service modules.	MEDIUM
2	**Dual charting libraries** — Both Chart.js and Recharts are included. Consider standardizing on one to reduce bundle size.	LOW
3	**Frontend middleware JWT decode** — Uses `atob` decode without signature verification. This is documented as intentional (UX-only), but worth noting for security reviews.	LOW
4	**Mixed route mounting** — Some routes are mounted at `/api/specificPath` while others use bare `/api` prefix (applications, chat, documents, milestones, news, appointments), making the API surface harder to discover.	LOW
5	**Auth controller size** — At 23 KB, it handles login, register, password reset, Google OAuth, and potentially more. Splitting into dedicated auth sub-controllers could improve maintainability.	MEDIUM

Luminedge CRM — Project Architecture Report

Generated on June 23, 2026 • Version 1.0.0 • Confidential

This report was auto-generated from the project source code analysis.